

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO CHỨC NĂNG GIỮA KỲ
MÔN: KIẾN TRÚC HƯỚNG DỊCH VỤ (SOA)
BOOKING SERVICE – THÀNH VIÊN 4

THÔNG TIN	NỘI DUNG
HỌ VÀ TÊN	Trần Trung Hiếu
MSSV	52400047
VAI TRÒ	Thành viên 4 – phát triển Booking Service bằng C#
NGÔN NGỮ	C# / .NET 8 (HttpListener + ADO.NET + System.Text.Json)
HTTP	Cổng 5004
DATABASE	MySQL – booking_db (có hỗ trợ SQL Server)

Báo cáo tập trung vào các sản phẩm P1–P8 của Booking Service.

MỤC LỤC

1. GIỚI THIỆU VÀ PHẠM VI	3
1.1. Công nghệ và lý do chọn	3
1.2. Phạm vi giữa kỳ và phần đề cuối kỳ	3
2. P1 – CƠ SỞ DỮ LIỆU + SEED	3
2.1. Bảng bookings	3
2.2. Schema và seed	4
3. P2 – ERD	4
4. P3 – USE CASE	5
5. P4 – DFD MỨC 0 VÀ MỨC 1	5
5.1. DFD mức 0	5
5.2. DFD mức 1	6
6. P5 – DFD MỨC 2	6
7. P6 – API, MÃ NGUỒN VÀ KIỂM THỬ	7
7.1. API	7
7.2. Xử lý chính trong mã nguồn	7
7.3. Cấu trúc mã nguồn	8
7.4. Kết quả chạy API	8
7.5. Kiểm thử tự động	10
8. P7 – TRANG ADMIN	11
9. P8 – TỔNG HỢP, HƯỚNG DẪN CHẠY VÀ TỰ KIỂM TRA	13
9.1. Chuẩn bị CSDL	13
9.2. Cấu hình Booking Service	13
9.3. Chạy service, trang admin và kiểm thử	13
9.4. Checklist P1–P8	13
10. KẾT LUẬN VÀ ĐỊNH HƯỚNG CUỐI KỲ	14
10.1. Các nội dung đã hoàn thành	14
10.2. Khó khăn gặp phải	14
10.3. Kế hoạch chuyển sang framework ở cuối kỳ	14
PHỤ LỤC – VIỆC CHUNG CỦA NHÓM DO TV4 PHỤ TRÁCH	14

1. GIỚI THIỆU VÀ PHẠM VI

Booking Service là service nhận và lưu đơn đặt vé trong hệ thống đặt vé xem phim theo kiến trúc microservices. Ở giữa kỳ, service tiếp nhận yêu cầu đặt vé, kiểm tra dữ liệu, kiểm tra ghế đã bị đặt, tính tổng tiền và lưu đơn với trạng thái PENDING vào CSDL riêng booking_db; đồng thời cho phép xem danh sách đơn và chi tiết một đơn.

Service gọi tới Booking: Client App (đặt vé, xem đơn) và Admin Dashboard (xem danh sách đơn). **Booking gọi tới (cuối kỳ):** Showtime Service để lấy giá vé, Payment Service để thanh toán và Notification Service để gửi xác nhận; khi đó Booking đóng vai trò Orchestrator. **Tham chiếu tới Booking:** transactions.bookingId của Payment và notification_logs.booking_id của Notification đều trỏ logic tới bookings.id.

1.1. Công nghệ và lý do chọn

Thành phần	Lựa chọn	Lý do
HTTP server	System.Net.HttpListener	Có sẵn trong .NET, không phải framework web; đúng yêu cầu "code thuần" giữa kỳ.
JSON	System.Text.Json	Có sẵn trong .NET; đọc body bằng JsonDocument để tự kiểm tra từng trường.
Truy cập CSDL	ADO.NET (DbProviderFactory) + MySqlConnection 2.3.7	Driver kết nối CSDL được phép theo Phụ lục C; dùng DbConnection chung nên đổi MySQL/SQL Server chỉ bằng biến DB_PROVIDER.
CSDL	MySQL 8 – booking_db	Thống nhất với các service khác của nhóm, chạy được bằng Docker; bản SQL Server (Microsoft.Data.SqlClient) để dùng trên Windows nếu cần.
Kiểm thử	Bộ test console tự viết	Không cần thư viện ngoài; gồm test đơn vị và test gọi API thật với MySQL.

1.2. Phạm vi giữa kỳ và phần để cuối kỳ

- **Đã triển khai:** POST /api/booking, GET /api/booking, GET /api/booking/{id}; envelope JSON thống nhất; CORS và OPTIONS 204; truy vấn có tham số; kiểm tra ghế trùng an toàn khi nhiều request cùng lúc; trang admin chỉ xem có nhãn màu trạng thái.
- **Chưa triển khai (cuối kỳ):** JWT (lấy username từ token), gọi Showtime lấy giá vé thay cho pricePerSeat client gửi, gọi Payment rồi Notification, cập nhật PAID/CANCELLED, xử lý timeout khi gọi service khác.

2. P1 – CƠ SỞ DỮ LIỆU + SEED

Booking Service áp dụng nguyên tắc database-per-service: chỉ sở hữu và thao tác trên booking_db. Các cột movie_id, showtime_id, username là khóa ngoại logic sang service khác, không tạo FOREIGN KEY vật lý.

2.1. Bảng bookings

Cột	Kiểu dữ liệu	Ràng buộc / ý nghĩa
id	INT	PRIMARY KEY, AUTO_INCREMENT (SQL Server: IDENTITY)
movie_id	INT	NOT NULL; FK logic → Movie.movies.id
showtime_id	INT	NOT NULL; FK logic → Showtime.showtimes.id
username	VARCHAR(50)	NOT NULL; FK logic → User.users.username
seats	VARCHAR(100)	NOT NULL; danh sách ghế, ví dụ "A1,A2"
total_price	INT	NOT NULL; số ghế × giá vé (VND); CHECK ≥ 0
status	VARCHAR(10)	NOT NULL DEFAULT 'PENDING'; CHECK ∈ {PENDING, PAID, CANCELLED}
created_at	DATETIME	NOT NULL; thời điểm tạo đơn (giờ máy chạy service)

Hai index hỗ trợ truy vấn: (showtime_id, status) cho bước kiểm tra ghế trùng và (username) cho tra cứu theo người đặt.

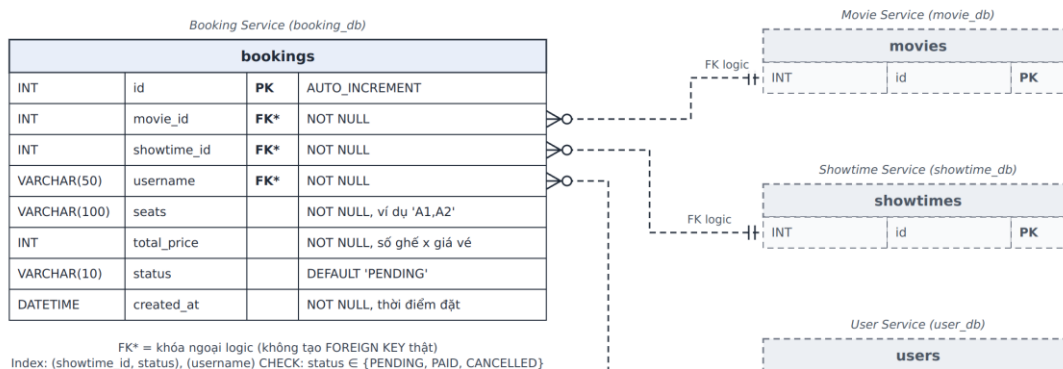
2.2. Schema và seed

- db/schema.sql dùng CREATE DATABASE IF NOT EXISTS và CREATE TABLE IF NOT EXISTS, không DROP/TRUNCATE nên chạy lại không mất đơn đã có; có SET NAMES utf8mb4 để tiếng Việt đúng khi chạy trên Windows.
- db/seed.sql nạp đúng 3 đơn id 1–3 của Phụ lục A.4 bằng REPLACE INTO: chạy lặp lại chỉ đưa 3 dòng mẫu về giá trị chuẩn, các đơn id ≥ 4 được giữ nguyên.
- Bản SQL Server nằm ở db/sqlserver/ (MERGE + IDENTITY_INSERT), cùng cấu trúc và dữ liệu.
- Đã kiểm tra: chạy schema.sql rồi seed.sql hai lần liên tiếp trên MySQL 8.0 không lỗi, dữ liệu khớp seed chung seed/booking_db.sql.

id	movie_id	showtime_id	username	seats	total_price	status
1	1	1	user1	A1,A2	180000	PAID
2	2	3	user2	B5	85000	CANCELLED
3	4	5	user1	C3,C4	180000	PENDING

3. P2 – ERD

ERD vẽ theo ký hiệu chân chim. Bảng bookings thuộc Booking Service; ba khóa ngoại logic vẽ nét đứt sang hộp mờ của Movie, Showtime và User Service. Mỗi đơn thuộc đúng một phim, một suất chiếu và một người dùng; một suất chiếu có thể có không hoặc nhiều đơn.

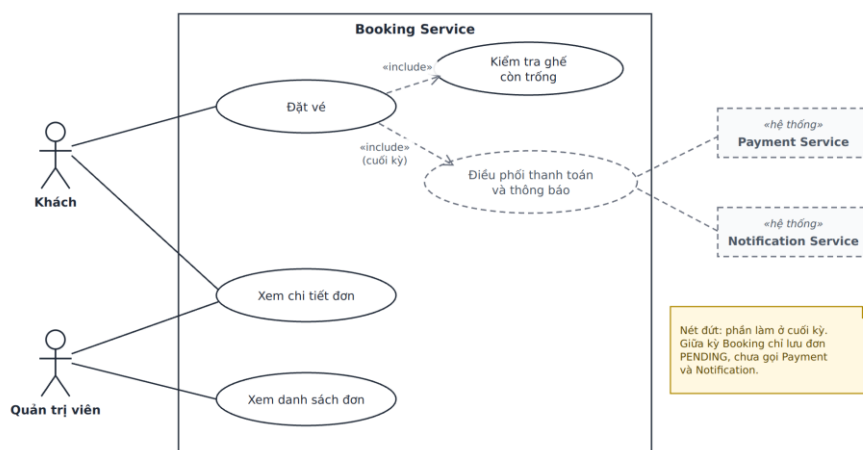


HÌNH P2 – ERD BOOKING SERVICE

Vì không có khóa ngoại thật, tính nhất quán do service gọi chịu trách nhiệm. Ở cuối kỳ, Booking sẽ hỏi Showtime Service để xác nhận suất chiếu tồn tại và lấy giá vé, thay vì dựa vào CSDL.

4. P3 – USE CASE

Khung hệ thống là Booking Service. Actor gồm Khách (đặt vé, xem đơn của mình), Quản trị viên (xem đơn) và hai actor hệ thống Payment Service, Notification Service cho phần điều phối ở cuối kỳ (vẽ nét đứt).



HÌNH P3 – USE CASE BOOKING SERVICE

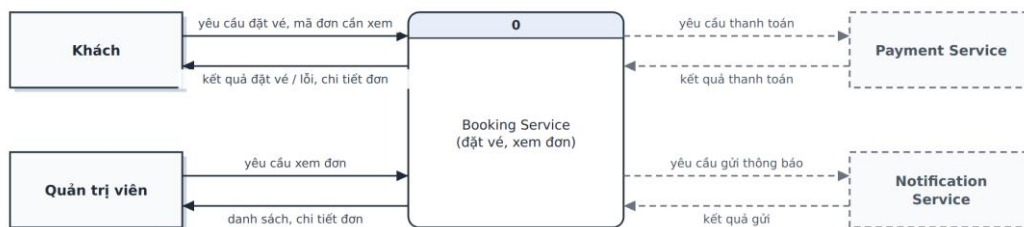
Use case	Actor	Luồng chính	Ngoại lệ
Đặt vé	Khách	- Khách gửi phim, suất chiếu, danh sách ghế, giá vé, tên đăng nhập. - Hệ thống kiểm tra dữ liệu. - Tính tổng tiền = số ghế × giá vé. - Kiểm tra ghế còn trống (include). - Lưu đơn PENDING, trả mã đơn và tổng tiền.	Thiếu/sai dữ liệu: báo lỗi, không lưu. Ghế đã có người đặt: báo danh sách ghế trùng.
Kiểm tra ghế còn trống	(được Đặt vé gọi)	Lấy ghế của các đơn cùng suất chiếu chưa bị hủy, so từng ghế với ghế khách chọn.	Đơn CANCELLED không giữ ghế.
Xem chi tiết đơn	Khách, Quản trị viên	Gửi mã đơn, hệ thống trả thông tin đơn.	Mã không hợp lệ hoặc không tồn tại: báo lỗi.

Use case	Actor	Luồng chính	Ngoại lệ
Xem danh sách đơn	Quản trị viên	Mở trang admin, hệ thống trả danh sách đơn; lọc theo trạng thái, người đặt, ghế.	Service chưa chạy: trang báo lỗi rõ ràng.
Điều phối thanh toán và thông báo (cuối kỳ)	Payment, Notification Service	Sau khi lưu PENDING, gọi Payment; PAID thì cập nhật đơn và gọi Notification; FAILED thì đơn thành CANCELLED.	Notification lỗi: đơn vẫn PAID, chỉ ghi log.

5. P4 – DFD MỨC 0 VÀ MỨC 1

5.1. DFD mức 0

Ở mức ngữ cảnh, Khách gửi yêu cầu đặt vé, mã đơn cần xem và nhận kết quả; Quản trị viên gửi yêu cầu xem đơn và nhận danh sách, chi tiết đơn. Payment và Notification Service vẽ nét đứt vì chỉ tham gia ở cuối kỳ.

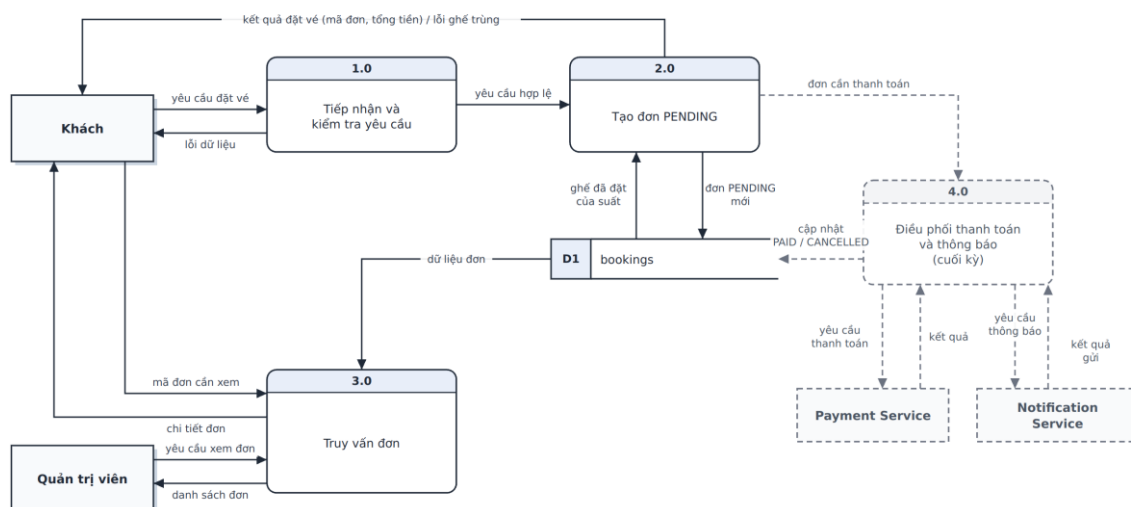


Nét đứt: thực thể và luồng dữ liệu thực hiện ở cuối kỳ

HÌNH P4.1 – DFD MỨC 0

5.2. DFD mức 1

Mức 1 phân rã Booking Service thành: 1.0 Tiếp nhận và kiểm tra yêu cầu; 2.0 Tạo đơn PENDING; 3.0 Truy vấn đơn; 4.0 Điều phối thanh toán và thông báo (cuối kỳ, nét đứt). Kho dữ liệu D1 là bookings. Các luồng với thực thể ngoài cân bằng với mức 0.

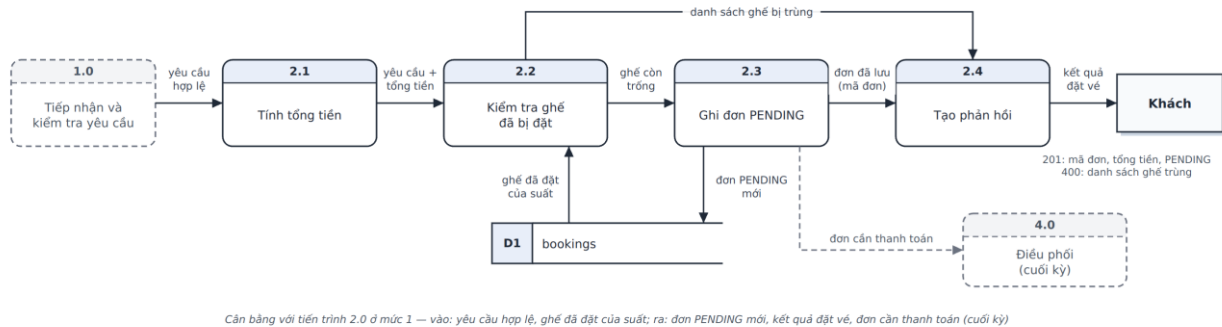


Nét đứt: thực hiện ở cuối kỳ (giữa kỳ chưa gọi Payment / Notification)

HÌNH P4.2 – DFD MỨC 1

6. P5 – DFD MỨC 2

Tiến trình 2.0 được phân rã thành 2.1 Tính tổng tiền; 2.2 Kiểm tra ghế đã bị đặt (đọc D1); 2.3 Ghi đơn PENDING (ghi D1); 2.4 Tạo phản hồi. Nếu 2.2 phát hiện ghế trùng, luồng "danh sách ghế bị trùng" đi thẳng tới 2.4 để trả lỗi 400. Luồng vào (yêu cầu hợp lệ, ghế đã đặt của suất) và luồng ra (đơn PENDING mới, kết quả đặt vé, đơn cần thanh toán ở cuối kỳ) cân bằng với tiến trình 2.0 ở mức 1. Trong mã nguồn, các bước này được đánh dấu bằng chú thích 2.1–2.4 trong BookingManager.cs.



HÌNH P5 – DFD MỨC 2 (PHÂN RÃ TIẾN TRÌNH 2.0)

7. P6 – API, MÃ NGUỒN VÀ KIỂM THỬ

7.1. API

Method & URL	Input	Output thành công	Mã lỗi
POST /api/booking	JSON {movieId, showtimeId, seats[], pricePerSeat, username}	201 + đơn {id, totalPrice, status: "PENDING", ...}; header Location	400 dữ liệu sai, JSON sai, ghế trùng; 413 body > 16 KiB; 503 suất chiếu bận; 500
GET /api/booking	—	200 + mảng đơn (mới nhất trước)	500
GET /api/booking/{id}	id nguyên dương	200 + một đơn	400 id sai; 404 không có; 500
OPTIONS (2 đường dẫn trên)	—	204 + header CORS	—
Method khác / đường dẫn khác	—	—	405 kèm header Allow / 404

Phản hồi thành công có dạng {"status": "success", "data": ...}, lỗi có dạng {"status": "error", "message": "...". Mọi phản hồi có header CORS: Allow-Origin *, Allow-Methods GET, POST, PUT, DELETE, OPTIONS, Allow-Headers Content-Type, Authorization.

7.2. Xử lý chính trong mã nguồn

- **Kiểm tra dữ liệu (1.0):** từng trường được kiểm tra riêng và gom lỗi vào một thông báo: id phải là số nguyên dương; 1–10 ghế, ghế dạng hàng A–Z + số 1–99, không lặp; pricePerSeat từ 1 đến 1.000.000; username 1–50 ký tự. Trường status, totalPrice do client gửi bị bỏ qua, server tự quyết định.

- **Kiểm tra ghế trùng (2.2):** lấy chuỗi ghế của các đơn cùng showtime_id có trạng thái khác CANCELLED, tách theo dấu phẩy rồi so từng ghế (không dùng LIKE) để A1 không bị nhầm với A10.
- **Chống đặt trùng khi nhiều request cùng lúc:** bước 2.2–2.3 chạy trong một transaction và giữ khóa theo tên cho từng suất chiếu (GET_LOCK trên MySQL, sp_getapplock trên SQL Server). Hai request đặt cùng ghế không thể cùng vượt qua bước kiểm tra.
- **An toàn:** mọi câu SQL dùng tham số; body tối đa 16 KiB; lỗi 500 chỉ trả thông báo chung, chi tiết ghi ở console; cấu hình CSDL nằm trong .env, không có mật khẩu trong mã nguồn.

7.3. Cấu trúc mã nguồn

Tệp (booking-service/src/)	Vai trò	DFD
BookingService/Program.cs	Điểm vào: đọc cấu hình, kiểm tra kết nối CSDL, mở cổng 5004	—
Config/AppConfig.cs	Đọc biến môi trường và file .env (PORT, DB_*)	—
Http/ApiServer.cs, HttpHelpers.cs	HttpListener, định tuyến, CORS, envelope JSON, đọc body	1.0, 2.4, 3.0
Domain/BookingRequestParser.cs	Kiểm tra từng trường của body	1.0
Domain/SeatRules.cs	Chuẩn hóa ghế, phát hiện ghế trùng	2.2
Services/BookingManager.cs	Nghiệp vụ tạo đơn: tính tiền, kiểm tra ghế, ghi đơn	2.1–2.4
Data/BookingRepository.cs, SqlDialect.cs, Database.cs	ADO.NET, truy vấn có tham số, transaction + khóa suất chiếu, khác biệt MySQL/SQL Server	2.3, 3.0, D1
BookingService.Tests/Program.cs	12 test đơn vị + 17 test gọi API thật	—

7.4. Kết quả chạy API

Các ảnh dưới đây là kết quả chạy thật trên môi trường kiểm thử (.NET 8.0.131, MySQL 8.0.46) với dữ liệu mẫu vừa nạp. Khi chạy service, console in cấu hình, phiên bản CSDL và một dòng log cho mỗi request.

```
Booking Service - Terminal (curl / mysql), môi trường kiểm thử

$ dotnet run
=== Booking Service (TV4 - C# HttpListener + ADO.NET) ===
Cấu hình : /home/claude/build/svc/.env
CSDL      : MySQL 127.0.0.1:3306/booking_db
Kết nối CSDL thành công (MySQL 8.0.46-0ubuntu0.24.04.4).
Lắng nghe: http://localhost:5004/ (API: http://localhost:5004/api/booking)
Nhấn Ctrl+C để dừng.
[2026-10-05 15:54:43] OPTIONS /api/booking -> 204 (0 ms)
[2026-10-05 15:54:43] GET      /api/booking -> 200 (6 ms)
[2026-10-05 15:54:43] POST     /api/booking -> 400 (5 ms)
[2026-10-05 15:54:43] POST     /api/booking -> 201 (7 ms)
[2026-10-05 15:54:43] POST     /api/booking -> 201 (6 ms)
[2026-10-05 15:54:43] GET      /api/booking/4 -> 200 (4 ms)
[2026-10-05 15:54:43] GET      /api/booking/99 -> 404 (2 ms)
[2026-10-05 15:54:43] POST     /api/booking -> 400 (1 ms)
[2026-10-05 15:54:43] POST     /api/booking -> 400 (1 ms)
[2026-10-05 15:54:43] DELETE   /api/booking/1 -> 405 (0 ms)
```

HÌNH P6.1 – KHỞI ĐỘNG SERVICE VÀ LOG REQUEST

Lệnh mẫu trong phiếu giao việc (đặt A1, A2 suất 1) trả 400 vì hai ghế này đã thuộc đơn #1 (PAID) trong dữ liệu mẫu, đúng quy tắc kiểm tra ghế. Đặt ghế khác (A3, A4) thì tạo được đơn #4 với tổng tiền $2 \times 90.000 = 180.000$.

Booking Service – Terminal (curl / mysql), môi trường kiểm thử

```
$ curl -i -X POST http://localhost:5004/api/booking -H "Content-Type: application/json" \
-d '{"movieId":1,"showtimeId":1,"seats":["A1","A2"],"pricePerSeat":90000,"username":"user1"}'
# (lược bớt header CORS/Date cho gọn)
HTTP/1.1 400 Bad Request

{
  "status": "error",
  "message": "Ghế đã có người đặt trong suất chiếu #1: A1, A2"
}

$ curl -i -X POST http://localhost:5004/api/booking -H "Content-Type: application/json" \
-d '{"movieId":1,"showtimeId":1,"seats":["A3","A4"],"pricePerSeat":90000,"username":"user1"}'
HTTP/1.1 201 Created
Location: /api/booking/4

{
  "status": "success",
  "data": {
    "id": 4,
    "movieId": 1,
    "showtimeId": 1,
    "username": "user1",
    "seats": [
      "A3",
      "A4"
    ],
    "totalPrice": 180000,
    "status": "PENDING",
    "createdAt": "2026-10-05T15:54:43"
  }
}
```

HÌNH P6.2 – POST GHẾ TRÙNG (400) VÀ POST THÀNH CÔNG (201)

Ghế B5 của suất 3 thuộc đơn #2 đã CANCELLED nên được đặt lại bình thường (đơn #5). Chi tiết đơn #4 và danh sách đơn đọc lại đúng dữ liệu đã lưu.

Booking Service – Terminal (curl / mysql), môi trường kiểm thử

```
$ curl -X POST http://localhost:5004/api/booking -H "Content-Type: application/json" \
-d '{"movieId":2,"showtimeId":3,"seats":["B5"],"pricePerSeat":85000,"username":"user2"}'

{
  "status": "success",
  "data": {
    "id": 5,
    "movieId": 2,
    "showtimeId": 3,
    "username": "user2",
    "seats": [
      "B5"
    ],
    "totalPrice": 85000,
    "status": "PENDING",
    "createdAt": "2026-10-05T15:54:43"
  }
}

$ curl -s http://localhost:5004/api/booking/4 | jq -c .
{"status":"success","data":{"id":4,"movieId":1,"showtimeId":1,"username":"user1","seats":["A3","A4"],"totalPrice":180000,"status":"PENDING","createdAt":"2026-10-05T15:54:43"}}

$ curl -s http://localhost:5004/api/booking | jq -c '.status, .data[]'
# (jq -c chỉ in gọn mỗi đơn một dòng)
"success"
{"id":5,"movieId":2,"showtimeId":3,"username":"user2","seats":["B5"],"totalPrice":85000,"status":"PENDING","createdAt":"2026-10-05T15:54:43"}
{"id":4,"movieId":1,"showtimeId":1,"username":"user1","seats":["A3","A4"],"totalPrice":180000,"status":"PENDING","createdAt":"2026-10-05T15:54:43"}
{"id":3,"movieId":4,"showtimeId":5,"username":"user1","seats":["C3","C4"],"totalPrice":180000,"status":"PENDING","createdAt":"2026-10-02T09:30:00"}
{"id":2,"movieId":2,"showtimeId":3,"username":"user2","seats":["B5"],"totalPrice":85000,"status":"CANCELLED","createdAt":"2026-10-01T11:00:00"}
{"id":1,"movieId":1,"showtimeId":1,"username":"user1","seats":["A1","A2"],"totalPrice":180000,"status":"PAID","createdAt":"2026-10-01T10:00:00"}
```

HÌNH P6.3 – ĐẶT LẠI GHẾ CỦA ĐƠN ĐÃ HỦY, GET CHI TIẾT VÀ GET DANH SÁCH

```
Booking Service - Terminal (curl / mysql), môi trường kiểm thử

$ curl -i -X OPTIONS http://localhost:5004/api/booking
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Max-Age: 600
Server: Microsoft-NetCore/2.0
Date: Mon, 05 Oct 2026 15:54:43 GMT
Content-Length: 0

$ curl -i http://localhost:5004/api/booking/99
# (lược bớt header CORS/Date cho gọn)
HTTP/1.1 404 Not Found

{
  "status": "error",
  "message": "Không tìm thấy đơn #99"
}

$ curl -X POST http://localhost:5004/api/booking -H "Content-Type: application/json" \
-d '{"movieId":0,"showtimeId":1,"seats":["A1","a1","K100"],"pricePerSeat":-5}'
{
  "status": "error",
  "message": "Dữ liệu không hợp lệ: movieId phải lớn hơn 0; Ghế A1 bị lặp trong yêu cầu; Ghế 'K100' không hợp lệ (dạng hàng A-Z + số 1-99, ví dụ A1); pricePerSeat phải lớn hơn 0; Thiếu trường username"
}

$ curl -X POST http://localhost:5004/api/booking -H "Content-Type: application/json" -d '{"movieId":1,'
{
  "status": "error",
  "message": "JSON không hợp lệ (dòng 1, vị trí 13)"
}

$ curl -i -X DELETE http://localhost:5004/api/booking/1
HTTP/1.1 405 Method Not Allowed
Allow: GET, OPTIONS

{
  "status": "error",
  "message": "Phương thức không được hỗ trợ. Chỉ chấp nhận: GET, OPTIONS"
}
```

HÌNH P6.4 – OPTIONS 204 VÀ CÁC TRƯỜNG HỢP LỖI 404, 400, 405

```
Booking Service - Terminal (curl / mysql), môi trường kiểm thử

$ mysql -u root -p -t -e "SELECT * FROM booking_db.bookings ORDER BY id;"
+-----+-----+-----+-----+-----+-----+-----+-----+
| id | movie_id | showtime_id | username | seats | total_price | status | created_at |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 1 | 1 | user1 | A1,A2 | 180000 | PAID | 2026-10-01 10:00:00 |
| 2 | 2 | 3 | user2 | B5 | 85000 | CANCELLED | 2026-10-01 11:00:00 |
| 3 | 4 | 5 | user1 | C3,C4 | 180000 | PENDING | 2026-10-02 09:30:00 |
| 4 | 1 | 1 | user1 | A3,A4 | 180000 | PENDING | 2026-10-05 15:54:43 |
| 5 | 2 | 3 | user2 | B5 | 85000 | PENDING | 2026-10-05 15:54:43 |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

HÌNH P6.5 – DỮ LIỆU THỰC TẾ TRONG BẢNG bookings

7.5. Kiểm thử tự động

Bộ kiểm thử BookingService.Tests chạy 12 test đơn vị (quy tắc ghế, kiểm tra body) và 17 test gọi API thật, trong đó có test gửi 8 request đặt cùng một ghế cùng lúc: đúng 1 request được 201, 7 request nhận 400. Test tự chọn ghế còn trống ở hàng Z nên chạy lặp lại được.

```

$ dotnet run --project src/BookingService.Tests
=== Kiểm thử Booking Service ===
--- U. Kiểm thử đơn vị ---
[PASS] U01 Chuẩn hóa ghế ' a1 ' -> 'A1'
[PASS] U02 Ghế hợp lệ A1, C12, Z99
[PASS] U03 Ghế sai A0, A100, AA1, 1A, rỗng
[PASS] U04 Tách chuỗi CSDL 'A1, a2, ' -> [A1,A2]
[PASS] U05 Ghế A1 không bị nhầm với A10 khi kiểm tra trùng
[PASS] U06 Không trùng ghế khi suất chiếu còn trống
[PASS] U07 Đọc request hợp lệ, chuẩn hóa ghế
[PASS] U08 Thiếu trường -> báo đủ 5 lỗi
[PASS] U09 Ghế lặp ['A1', 'a1'] bị từ chối
[PASS] U10 Quá 10 ghế bị từ chối
[PASS] U11 pricePerSeat = 0, movieId = "1", showtimeId = 1.5 bị từ chối
[PASS] U12 Body không phải object bị từ chối
--- A. Kiểm thử API thật tại http://localhost:5004 ---
[PASS] A01 OPTIONS /api/booking -> 204 + CORS
[PASS] A02 GET /api/booking -> 200, có đơn mẫu 1-3
[PASS] A03 GET /api/booking/1 -> 200, đúng dữ liệu mẫu
[PASS] A04 GET /api/booking/999999 -> 404
[PASS] A05 GET /api/booking/abc -> 400
[PASS] A06 POST đặt Z1,Z2 -> 201, PENDING, tổng tiền = 2 x 75000
[PASS] A07 GET đơn vừa tạo -> 200, dữ liệu đã lưu trong CSDL
[PASS] A08 POST trùng ghế Z2 -> 400
[PASS] A09 Ghế B5 của đơn #2 đã CANCELLED (suất 3) được đặt lại -> 201
[PASS] A10 POST JSON sai cú pháp -> 400
[PASS] A11 POST body rỗng -> 400
[PASS] A12 POST thiếu trường -> 400 kèm thông báo
[PASS] A13 PUT /api/booking -> 405 + header Allow
[PASS] A14 DELETE /api/booking/1 -> 405
[PASS] A15 GET /api/khong-ton-tai -> 404
[PASS] A16 POST body 20 Kib -> 413
[PASS] A17 8 request đặt cùng ghế Z3 cùng lúc -> đúng 1 thành công

Kết quả: 29 PASS, 0 FAIL, 0 SKIP

```

HÌNH P6.6 – KẾT QUẢ KIỂM THỬ: 29 PASS, 0 FAIL

Kiểm tra	Kết quả đã ghi nhận
dotnet build (Release)	PASS – 0 warning, 0 error
db/schema.sql + db/seed.sql chạy 2 lần	PASS – dữ liệu khớp Phụ lục A.4
BookingService.Tests trên CSDL mới	29 PASS, 0 FAIL, 0 SKIP
Chạy lại bộ test lần 2	28 PASS, 1 SKIP (ghế B5 đã đặt lại ở lần 1, đúng thiết kế)
Trang admin trên Chromium	Hiển thị dữ liệu thật; lọc; xem chi tiết; báo lỗi khi API tắt

Giới hạn cần ghi rõ: ảnh được chụp trên môi trường kiểm thử Linux; cần chạy lại bộ test và chụp ảnh trên máy demo (Windows) trước buổi kiểm tra. Nhánh SQL Server đã viết theo cùng cấu trúc nhưng chưa chạy thử với SQL Server thật.

8. P7 – TRANG ADMIN

Trang admin/pages/admin_booking.html là trang chỉ xem (HTML + CSS + JS thuần, một file). Đầu file có hằng API_BASE = "http://localhost:5004" để dễ ghép vào admin chung 6 tab. Trang gọi GET /api/booking để hiển thị bảng đơn và gọi GET /api/booking/{id} khi bấm "Xem".

Đơn đặt vé

Theo dõi các đơn khách đã đặt và trạng thái thanh toán của từng đơn.

Tải lại

Tất cả 5

PENDING 3

PAID 1

CANCELLED 1

Tìm theo người đặt, mã đơn hoặc ghế (ví dụ user1, 3, A1)

Đã thanh toán: 180.000 đ

Hiện thị 5 / 5 đơn.

Mã đơn	Người đặt	Phim	Suất chiếu	Ghế	Tổng tiền	Trạng thái	Thời gian đặt	
#5	user2	Phim #2	Suất #3	B5	85.000 đ	PENDING	05/10/2026 15:54	Xem
#4	user1	Phim #1	Suất #1	A3 A4	180.000 đ	PENDING	05/10/2026 15:54	Xem
#3	user1	Phim #4	Suất #5	C3 C4	180.000 đ	PENDING	02/10/2026 09:30	Xem
#2	user2	Phim #2	Suất #3	B5	85.000 đ	CANCELLED	01/10/2026 11:00	Xem
#1	user1	Phim #1	Suất #1	A1 A2	180.000 đ	PAID	01/10/2026 10:00	Xem

Nguồn API: http://localhost:5004/api/booking.

Trang chỉ xem, không sửa dữ liệu.

HÌNH P7.1 – DANH SÁCH ĐƠN VỚI NHÃN MÀU TRẠNG THÁI

Chi tiết đơn #4

Đóng

Dữ liệu lấy từ GET /api/booking/4

Mã đơn

#4

Người đặt

user1

Mã phim

#1

Mã suất chiếu

#1

Ghế

A3 A4

Số ghế

2

Tổng tiền

180.000 đ

Trạng thái

PENDING

Ý nghĩa

Chờ thanh toán

Thời gian đặt

05/10/2026 15:54

Tìm theo người đặt, mã đơn hoặc ghế (ví dụ user1, 3, A1)

Đã thanh toán: 180.000 đ

Hiện thị 5 / 5 đơn.

Mã đơn	Người đặt	Phim	Suất chiếu	Ghế	Tổng tiền	Trạng thái	Thời gian đặt	
#5	user2	Phim #2	Suất #3	B5	85.000 đ	PENDING	05/10/2026 15:54	Xem
#4	user1	Phim #1	Suất #1	A3 A4	180.000 đ	PENDING	05/10/2026 15:54	Xem
#3	user1	Phim #4	Suất #5	C3 C4	180.000 đ	PENDING	02/10/2026 09:30	Xem
#2	user2	Phim #2	Suất #3	B5	85.000 đ	CANCELLED	01/10/2026 11:00	Xem
#1	user1	Phim #1	Suất #1	A1 A2	180.000 đ	PAID	01/10/2026 10:00	Xem

Nguồn API: http://localhost:5004/api/booking.

Trang chỉ xem, không sửa dữ liệu.

HÌNH P7.2 – NGĂN CHI TIẾT ĐƠN (GET /api/booking/{id})

Thành phần	Mô tả
Bảng đơn	Mã đơn, người đặt, phim, suất chiếu, ghế, tổng tiền, trạng thái, thời gian đặt
Nhãn trạng thái	PENDING vàng, PAID xanh, CANCELLED đỏ (theo phiếu giao việc)
Lọc	Theo trạng thái (kèm số đơn mỗi loại) và ô tìm theo người đặt, mã đơn, ghế
Tải lại / chi tiết	Nút Tải lại gọi lại API; nút Xem mở ngăn chi tiết đơn
Trạng thái	Đang tải / không có dữ liệu / không khớp bộ lọc / lỗi kết nối
An toàn hiển thị	Dùng textContent, không chèn HTML từ dữ liệu API; class có tiền tố bk- để không đụng CSS khi ghép tab

12

Đơn đặt vé

Theo dõi các đơn khách đã đặt và trạng thái thanh toán của từng đơn.

Tải lại

Tất cả 0

PENDING 0

PAID 0

CANCELLED 0

Tìm theo người đặt, mã đơn hoặc ghế (ví dụ user1, 3, A1)

Đã thanh toán: 0 đ

Không kết nối được Booking Service tại http://localhost:5004. Hãy chạy service (dotnet run trong thư mục booking-service) rồi bấm Tải lại.

Chưa có đơn đặt vé nào. Đơn mới xuất hiện ở đây sau khi gọi POST /api/booking.

Không có dữ liệu để hiển thị.

HÌNH P7.3 – THÔNG BÁO KHI BOOKING SERVICE CHƯA CHẠY

9. P8 – TỔNG HỢP, HƯỚNG DẪN CHẠY VÀ TỰ KIỂM TRA

9.1. Chuẩn bị CSDL

Cách 1 – Docker (MySQL 8, cổng 3307 để không trùng MySQL của service khác), tại thư mục booking-service:

```
copy .env.example .env      (sửa DB_PASSWORD trong .env)
docker compose up -d        (lần đầu tự chạy schema.sql và seed.sql)
```

Cách 2 – MySQL có sẵn (XAMPP, MySQL Installer): sửa DB_PORT=3306 trong .env rồi chạy:

```
mysql -u root -p -e "source db/schema.sql"
mysql -u root -p -e "source db/seed.sql"
```

Cách 3 – SQL Server: đặt DB_PROVIDER=sqlserver và chạy hai file trong db/sqlserver/ bằng SSMS.

9.2. Cấu hình Booking Service

Biến môi trường	Giá trị mẫu	Ý nghĩa
PORT	5004	Cổng HTTP
LISTEN_PREFIXES	http://localhost:5004/	Prefix HttpListener; dùng http://+:5004/ khi ghép qua LAN
DB_PROVIDER	mysql	mysql hoặc sqlserver
DB_HOST / DB_PORT	127.0.0.1 / 3307	Địa chỉ CSDL
DB_NAME	booking_db	Tên CSDL
DB_USER / DB_PASSWORD	tài khoản DB local	Không đưa mật khẩu thật vào báo cáo và Git

Service kiểm tra kết nối CSDL khi khởi động (thứ 5 lần); nếu không kết nối được thì in hướng dẫn và thoát.

9.3. Chạy service, trang admin và kiểm thử

```
cd booking-service\src\BookingService
dotnet run

cd soa-datve      (thư mục gốc repo, cửa sổ khác)
python -m http.server 8081 --bind 127.0.0.1 --directory admin/pages
```

```
-> mở http://localhost:8081/admin_booking.html

cd booking-service
dotnet run --project src/BookingService.Tests
```

Trên Windows PowerShell dùng curl.exe thay cho curl khi gọi API bằng tay.

9.4. Checklist P1–P8

Mục	Nội dung	Trạng thái
P1	CSDL + seed (MySQL, SQL Server)	☒
P2	ERD_booking.png + .drawio	☒
P3	UC_booking.png + .drawio	☒
P4	DFD0_booking, DFD1_booking (.png + .drawio)	☒
P5	DFD2_booking (.png + .drawio)	☒
P6	API code thuần + test 29/29 + ảnh chạy	☒ ; chụp lại trên máy demo
P7	admin_booking.html	☒
P8	05_baocao_booking.docx	☒

10. KẾT LUẬN VÀ ĐỊNH HƯỚNG CUỐI KỲ

10.1. Các nội dung đã hoàn thành

- Thiết kế bảng bookings, schema và seed chạy lặp lại được cho MySQL và SQL Server.
- ERD, Use Case, DFD mức 0/1 và DFD mức 2 (phân rã tiến trình tạo đơn).
- Ba endpoint bằng HttpListener + ADO.NET, kiểm tra dữ liệu, kiểm tra ghế trùng an toàn khi nhiều request cùng lúc, CORS/OPTIONS, xử lý lỗi 400/404/405/413/500.
- Bộ kiểm thử 29 trường hợp chạy với CSDL thật; trang admin có nhãn trạng thái, lọc và xem chi tiết.

10.2. Khó khăn gặp phải

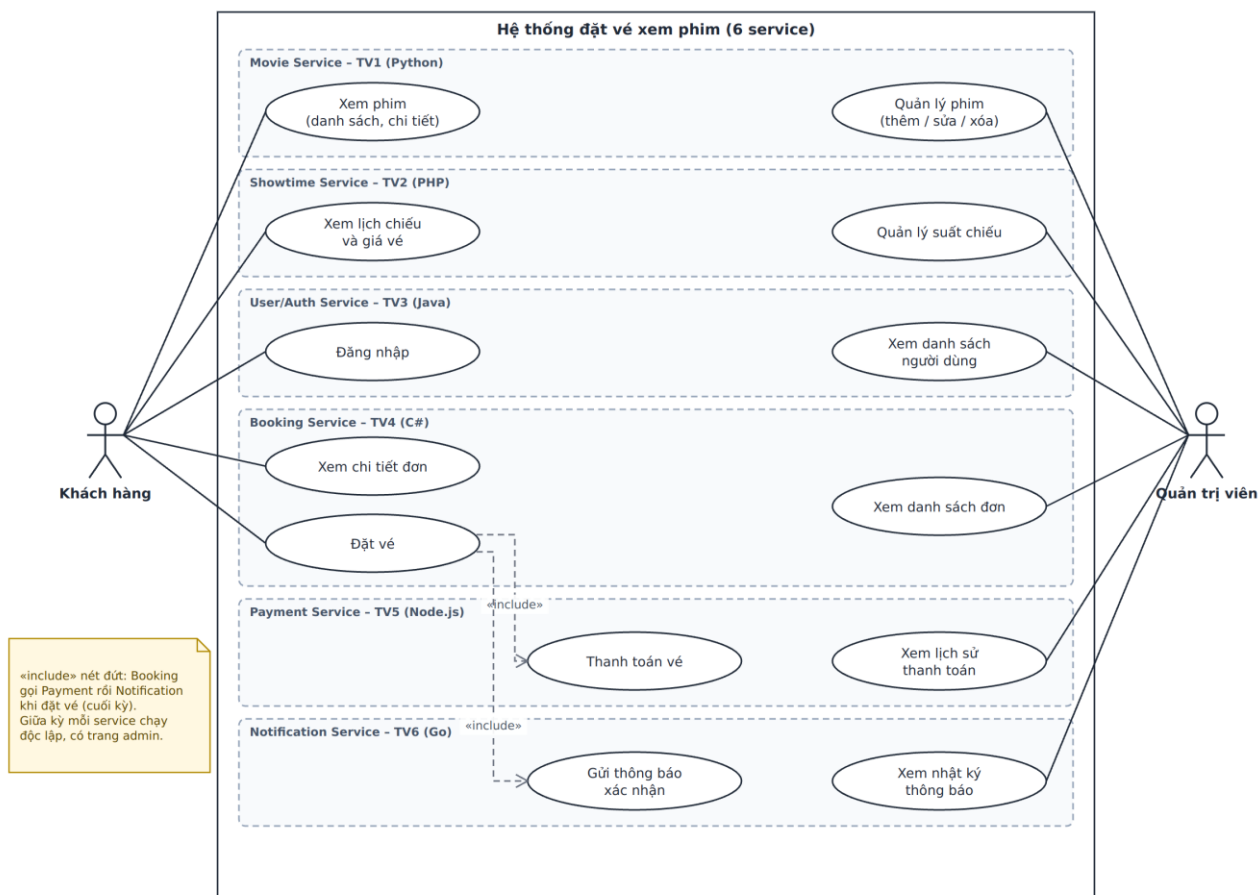
- HttpListener trên Windows chỉ nhận prefix http://localhost:5004/ khi không có quyền Administrator; muốn máy khác gọi được phải đăng ký netsh http add urlacl một lần.
- Kiểm tra ghế rồi mới ghi đơn có thể bị hai request chen nhau; giải quyết bằng transaction và khóa theo suất chiều, kiểm chứng bằng test 8 request đồng thời.
- Cột seats lưu dạng chuỗi "A1,A2" nên phải tách chuỗi trong code để so ghế chính xác.

10.3. Kế hoạch chuyển sang framework ở cuối kỳ

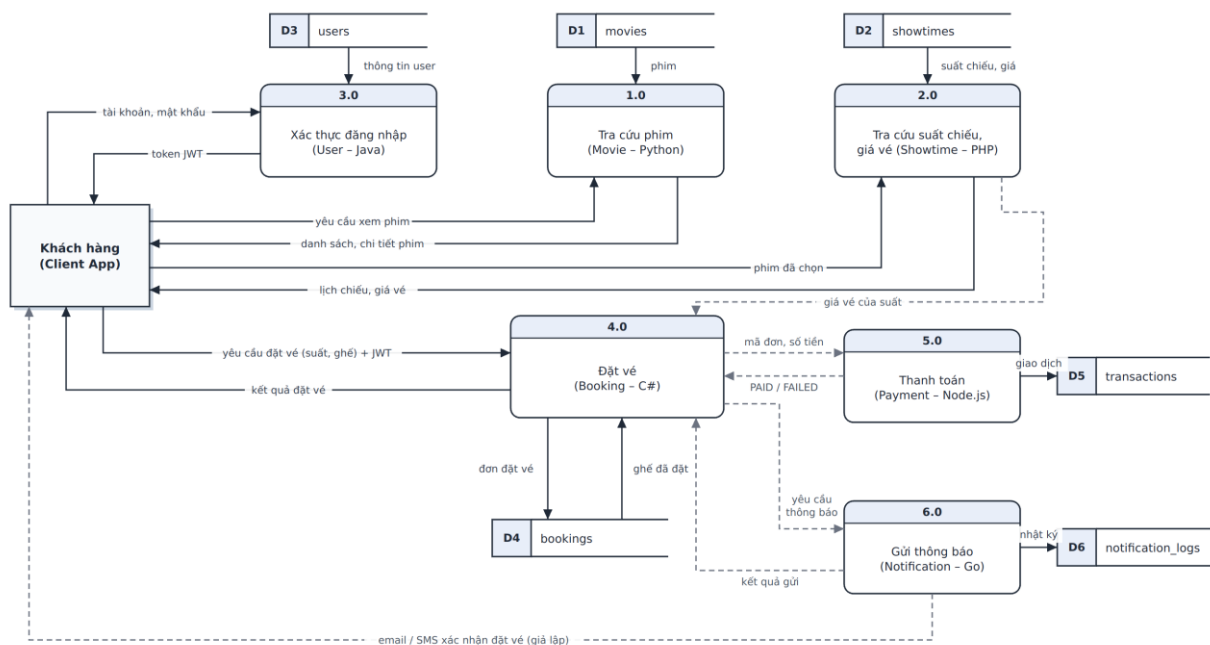
- Chuyển sang ASP.NET Core 8 Minimal API + EF Core; giữ nguyên hợp đồng API và tách lớp nghiệp vụ đã có (BookingManager, SeatRules) để dùng lại.
- Thêm JWT middleware (HS256, cùng khóa với User Service): lấy username từ token thay vì body.
- Dùng HttpClientFactory gọi Showtime lấy giá vé, gọi Payment rồi Notification (kèm X-Internal-Key), cập nhật PAID/CANCELLED, đặt timeout; Notification lỗi thì đơn vẫn PAID.

PHỤ LỤC – VIỆC CHUNG CỦA NHÓM DO TV4 PHỤ TRÁCH

Theo phiếu giao việc, TV4 ghép Use Case tổng thể và DFD mức 1 tổng thể cho luồng đặt vé. Hai sơ đồ dưới đây dựng theo use case và luồng gợi ý trong phiếu riêng của 6 thành viên và Kế hoạch v2.1; cần đối chiếu lại với sơ đồ thực tế của từng bạn trước khi đưa vào báo cáo tổng quan.



HÌNH PL.1 – USE CASE TỔNG THỂ HỆ THỐNG



Nét đứt: Booking (4.0) điều phối Showtime, Payment, Notification ở cuối kỳ. Luồng quản trị của Quản trị viên xem DFD mức 1 của từng service.

HÌNH PL.2 – DFD MỨC 1 TỔNG THỂ (LUỒNG ĐẶT VÉ)

Luồng đặt vé ở cuối kỳ: Khách đăng nhập (3.0) nhận JWT, xem phim (1.0) và suất chiếu (2.0), gửi yêu cầu đặt vé tới Booking (4.0). Booking kiểm tra ghế, lưu đơn PENDING (D4), lấy giá từ Showtime, gọi Payment (5.0); nếu PAID thì gọi Notification (6.0) để gửi xác nhận. Ở giữa kỳ các service chạy độc lập, chưa gọi chéo (các luồng nét đứt).

— HẾT —